

CackalackyCon 2026

AI Village – Build Your Own Security Agent

May 15–17 · Durham, NC

*Last year we watched AI demos. This year we build with it.
AI Village Coordinators: Leo and Xenia*

Agenda

01

What is an AI Agent?

05

Key Technologies

02

Why Security Agents?

06

Workshop: The Three Levels

03

Agent Design Architectures

04

Agent Memory

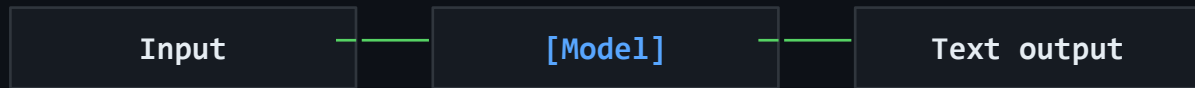


What is an AI Agent?

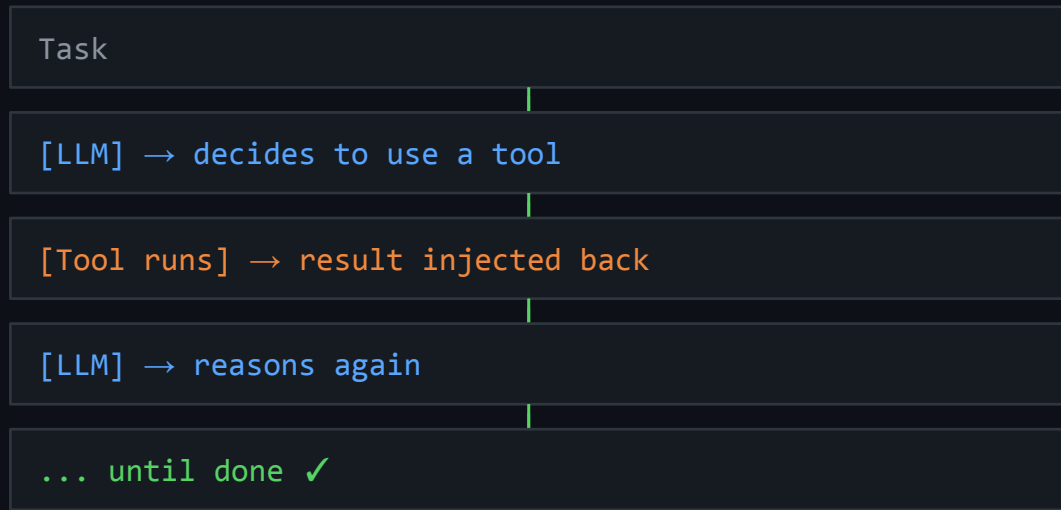
From chatbot to autonomous system

LLM vs. Agent

A plain LLM



An agent adds a loop



The agent can act on the world — read files, run code, search the web.

Key Properties of an Agent



Autonomy

Decides what steps to take without explicit instructions for each one



Tool Use

Calls real functions: read files, execute code, query databases



Reasoning

Plans multi-step paths toward a goal



Memory

Retains context across steps and optionally across sessions



Goal-Directed

Given an objective, figures out how to accomplish it



Why Security Agents?

Where the loop actually helps

Security Work is a Good Fit

Security tasks tend to be:

Repetitive

Parsing logs, correlating CVEs, running playbook steps

Multi-step

Recon → exploit research → remediation chains many lookups

Time-sensitive

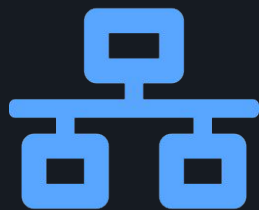
IR triage at 2am benefits from something that never sleeps

Knowledge-intensive

CVEs by version, TTPs by pattern

Agents are good at:

- Log analysis and anomaly detection
- IOC enrichment — IP, domain, hash lookups
- CVE correlation with discovered service versions
- Incident triage and response playbook generation
- CTF challenge solving — encoding, OSINT, forensics

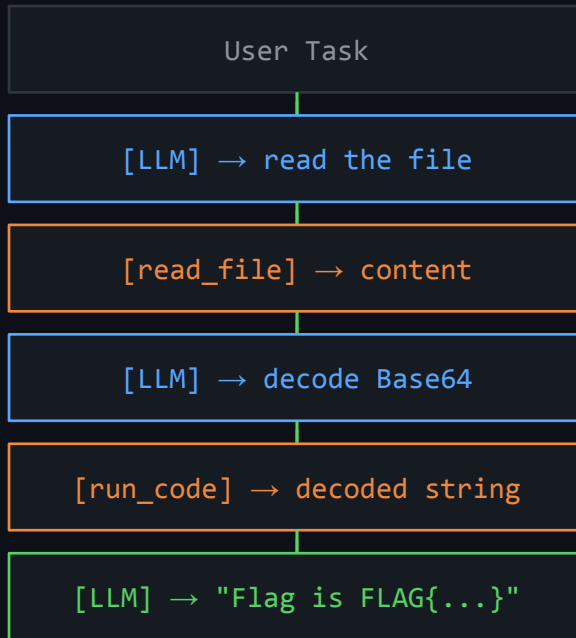


Agent Architectures

How you wire the pieces together

Architecture 1 – Single ReAct Agent

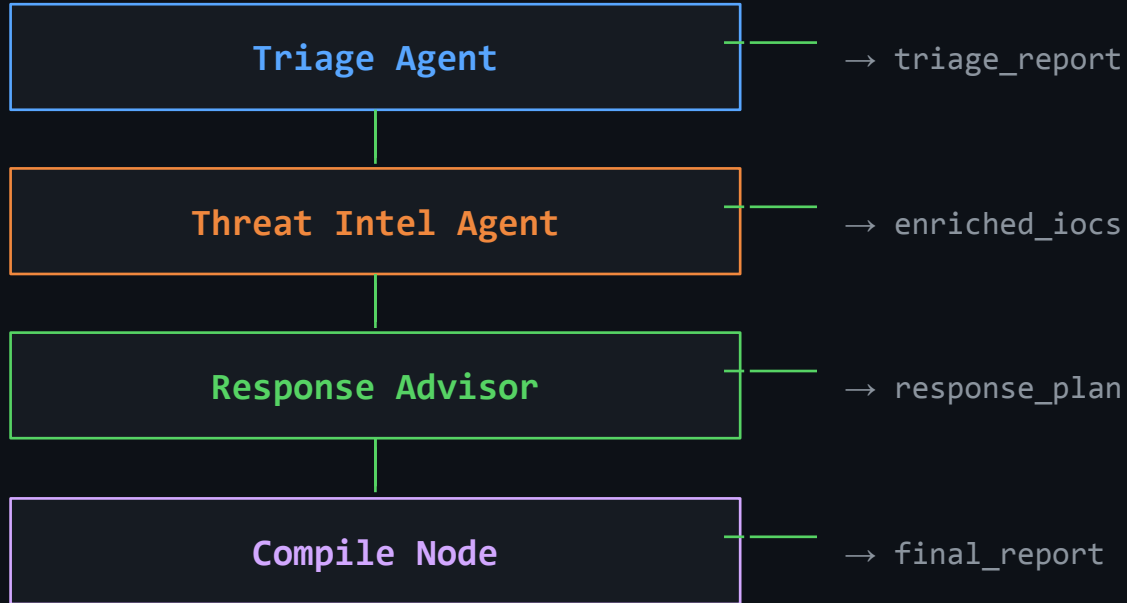
One LLM + a toolbox. Reasons, calls tools, observes results, loops.



Used in Level 1 and Level 2

Architecture 2 – Sequential Pipeline

Multiple specialized agents, each doing one job, passing results via shared state.



Why: each agent has a focused system prompt → better results from small models.



Agent Memory

What the agent remembers, and for how long

The Four Types of Memory



In-context

Scope: Current run only

Full message history in the LLM's context window



External / Episodic

Scope: Across runs

A vector database of past incident reports



Procedural

Scope: Baked into weights

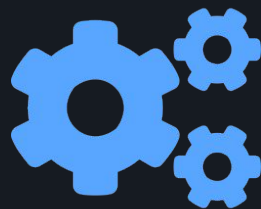
Everything the LLM learned during training



Semantic / State

Scope: Shared between agents

The SOCState TypedDict passed between L3 agents



Key Technologies

Ollama • LangChain • LangGraph

Ollama — Local LLMs, No Cloud

Runs open-weight models entirely on your laptop. No API key. No cloud. No data leaving your machine.

```
ollama pull llama3.1:8b    # primary model   - ~4.7 GB
ollama pull llama3.2:3b    # multi-agent    - ~2.0 GB
ollama serve                # start the local API server
```

Model	RAM Needed	Use
llama3.1:8b	8 GB+	L1, L2, L3 primary agent
llama3.2:3b	4 GB+	L3 multi-agent, slow machines

The ReAct pattern: Reason → Act → Observe. The docstring on your @tool function is the agent's only documentation for that tool.

LangChain & LangGraph

LangChain — Agent Framework

```
from langchain_core.tools import tool

@tool
def read_file(file_path: str) -> str:
    """Read a challenge file.
    LLM reads this docstring to decide
    when to call this tool."""
    ...
```

LangGraph — Multi-Agent Graphs

```
workflow = StateGraph(SOCState)

workflow.add_node("triage",
                  triage_agent)
workflow.add_node("threat_intel",
                  threat_intel_agent)

workflow.add_edge("triage",
                  "threat_intel")

graph = workflow.compile()
```

Rule of thumb:

The docstring on your @tool function is the LLM's only documentation for that tool. Write it clearly or the agent won't use it correctly.



The Workshop

Three levels, one afternoon

Workshop Levels

L1

The Solo Agent

Beginner | `level1/agent.py`

Single agent with file-reader and code-execution tools. Solve CTF challenges: decoding, OSINT, multi-layer obfuscation.

L2A

The Recon Agent

Intermediate | `level2/challenges/recon/agent.py`

Multi-step agent that analyzes pre-captured nmap scans, HTTP headers, DNS records, and correlates CVEs — you write the system and user prompts.

L2B

The Malware Analyst

Intermediate | `level2/challenges/malware/agent.py`

Multi-step agent that reads malware samples, deobfuscates payloads, and maps behavior to MITRE ATT&CK techniques.

L3

The SOC Squad

Advanced | `level3/soc_squad.py`

Three agents in a LangGraph pipeline: Triage → Threat Intel → Response Advisor. Produce a complete IR report.

Workshop Levels

L1

The Solo Agent

Beginner | `level1/agent.py`

CTF-solving agent with `read_file` + `run_code` tools. Complete 4 challenges: encoding, OSINT, log analysis, and a classical cipher.

L2

The Recon Agent

Intermediate | `level2/agent.py`

Recon agent with `parse_nmap`, `analyze_headers`, `check_cve`, `search_web`. Write the system prompt and user message from scratch.

L3

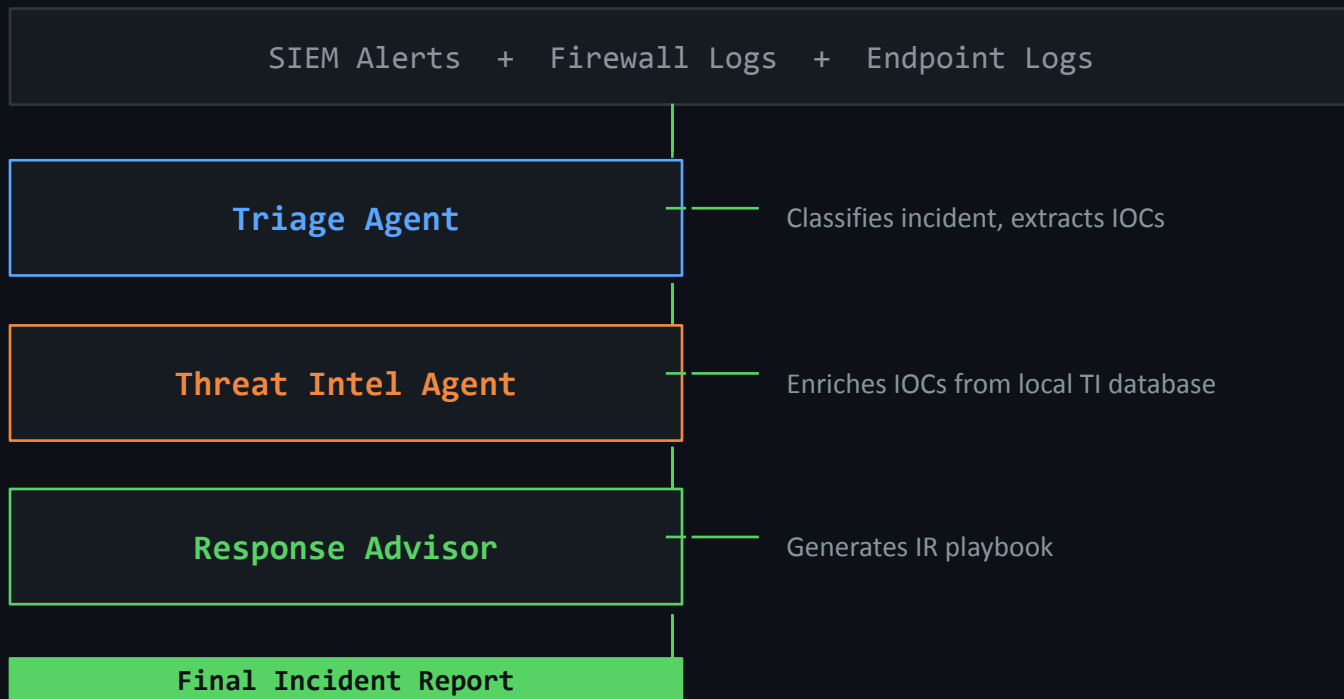
The SOC Squad

Advanced | `level3/soc_squad.py`

Three agents in a LangGraph pipeline: Triage → Threat Intel → Response Advisor. Produce a complete IR report.

Level 3 – SOC Squad Pipeline

Three agents collaborating on incident response via LangGraph:



Setup Check

Run this — raise your hand if anything is red:

```
uv run python verify.py
```

```
[✓] Ollama is running  
[✓] llama3.1:8b model available  
[✓] llama3.2:3b model available  
[✓] LangChain installed  
[✓] All challenge files present  
🎉 You're ready for CackalackyCon 2026!
```

Ollama not running?

→ open a second terminal: `ollama serve`

Model too slow?

→ change `llama3.1:8b` to `llama3.2:3b` in agent file

Let's Build

```
level1/agent.py    ← start here  
level2/agent.py    ← when you're ready  
level3/soc_squad.py ← final boss
```

Flags are always: `FLAG{some_text_here}`

Instructors are walking around. Ask questions.

Go.



<https://github.com/mundruid/cackalackycon-ai-village-2026>



Flags are always: `FLAG{some_text_here}`

Instructors are walking around. Ask questions.

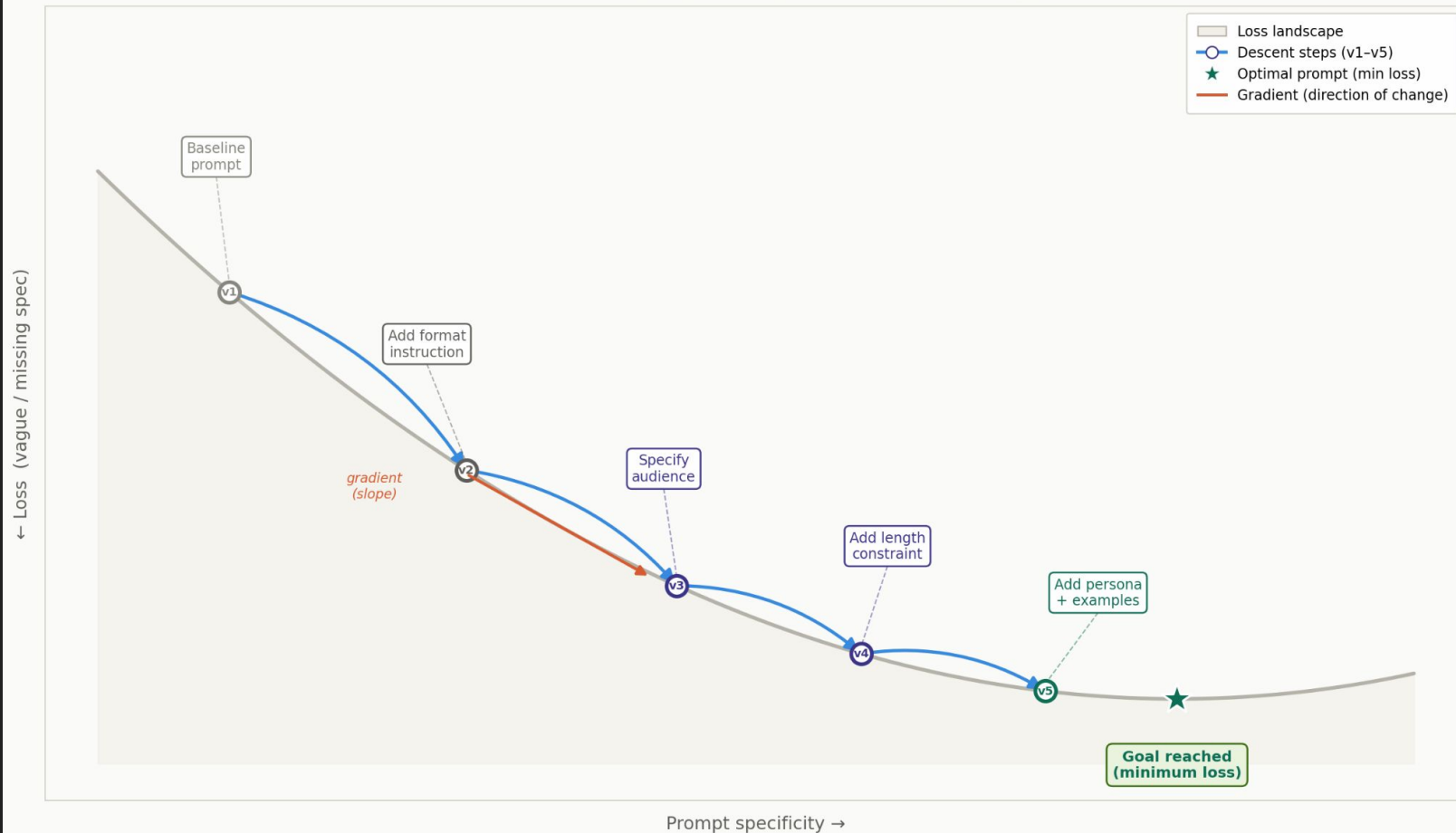
Go.

Tips for creativity

- **Think** about a problem
- **Decide** which tool to use
- **Act** (call the tool)
- **Observe** the result
- **Repeat** until it finds the flag

Prompt Gradient Descent

Each iteration refines the prompt, reducing loss toward a well-specified goal



Tips for creativity

- Make small changes and experiment
- Your prompt is a gradient descent
- Is it better to change your system prompt or the individual challenge prompt or both?
- One change at a time or more than one?
- Are tools used or is the model doing its own thing?
- Are tools overused?
- What is the difference between planning mode vs acting mode?